

Tool implementation — amounts.py

src/lab_billing/reconstruct/amounts.py — git blob b4e435ad1402 — v1.4.0 (2026-09-22)

```
"""Amount engine (module A, v1.0 — spec section 08).

Tracks gross posted, reversed, net observed posted and unallocated amounts.
Allocation sums exceeding the payment amount block the transaction's math
and record a conflict. Partial reversals are applied by their exact amount
and never double-deducted. Cross-currency amounts are never summed.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

@dataclass
class LineageAmounts:
    lineage_id: str
    gross_posted: int = 0
    reversed: int = 0
    net_observed_posted: int = 0
    unallocated: int = 0
    blocked_transactions: list[str] = field(default_factory=list)

@dataclass
class AmountConflict:
    kind: str
    detail: str
    event_ids: list[str]

class AmountEngine:
    def __init__(self, run_id: str):
        self.run_id = run_id
        self.conflicts: list[AmountConflict] = []

    def compute(self, events: list[dict], allocations: list[dict],
                lineage_of_record: dict[str, str] -> dict[str, LineageAmounts]:
        """Returns lineage -> amounts. Only USD rows are counted; other
        currencies are reported as conflicts, never converted."""
        out: dict[str, LineageAmounts] = {}
        payment_amount: dict[str, int] = {}
        allocation_sum: dict[str, int] = {}
        reversal_targets: dict[str, list[dict]] = {}
        reversals_unresolved: list[dict] = []

        for e in events:
            if e.get("currency") not in ("", "USD"):
                self.conflicts.append(AmountConflict(
                    "NON_USD_EVENT", f"event {e['event_id']} currency {e.get('currency')}
excluded",
                    [e["event_id"]]))
                continue
```

```

        if e["event_type"] == "PAYMENT_POSTED":
            amt = e.get("amount_minor") or 0
            if amt > 0:
                payment_amount[e["event_id"]] = amt
            elif e["event_type"] == "PAYMENT_REVERSED":
                rev = (e.get("reverses_event_id") or "").strip()
                if rev:
                    reversal_targets.setdefault(rev, []).append(e)
                else:
                    reversals_unresolved.append(e)

payment_to_bills: dict[str, list[str]] = {}
for a in allocations:
    if a.get("currency") != "USD":
        self.conflicts.append(AmountConflict(
            "NON_USD_ALLOCATION", f"allocation {a['allocation_id']} excluded",
            [a["allocation_id"]]))
        continue
    peid = a["payment_event_id"]
    amt = a.get("allocated_amount_minor") or 0
    allocation_sum[peid] = allocation_sum.get(peid, 0) + amt
    bid = (a.get("bill_record_id") or "").strip()
    if bid:
        payment_to_bills.setdefault(peid, []).append(bid)

# over-allocation blocks the transaction
blocked = [peid for peid, amt in payment_amount.items()
            if allocation_sum.get(peid, 0) > amt]
for peid in blocked:
    self.conflicts.append(AmountConflict(
        "OVER_ALLOCATION",
        f"payment {peid} amount {payment_amount[peid]} "
        f"< allocation sum {allocation_sum[peid]}",
        [peid]))
    payment_amount.pop(peid, None)
    allocation_sum.pop(peid, None)

def lin_of_event(eid: str) -> str:
    ev = next((x for x in events if x["event_id"] == eid), None)
    if ev and ev.get("bill_record_id"):
        return lineage_of_record.get(ev["bill_record_id"], "UNLINKED")
    # unallocated-to-bill payments attach through their allocations
    for bid in payment_to_bills.get(eid, []):
        lin = lineage_of_record.get(bid)
        if lin:
            return lin
    return "UNLINKED"

# gross posted and reversals per lineage
for e in events:
    if e["event_type"] != "PAYMENT_POSTED":
        continue
    peid = e["event_id"]
    amt = payment_amount.get(peid, 0)
    if amt <= 0:
        continue
    lin = lin_of_event(peid)
    acc = out.setdefault(lin, LineageAmounts(lineage_id=lin))

```

```

    acc.gross_posted += amt
    allocated = allocation_sum.get(peid, 0)
    if allocated < amt:
        acc.unallocated += amt - allocated
    revs = reversal_targets.get(peid, [])
    for r in revs:
        r_amt = r.get("amount_minor") or 0
        remaining = max(0, amt - acc.reversed)
        if remaining == 0:
            break # cumulative reversal cap reached
        acc.reversed += min(r_amt, remaining)

for r in reversals_unresolved:
    self.conflicts.append(AmountConflict(
        "UNRESOLVED_REVERSAL",
        f"reversal {r['event_id']} has no target; not deducted anywhere",
        [r["event_id"]]))

for acc in out.values():
    acc.net_observed_posted = acc.gross_posted - acc.reversed
return out

```